

Chap.4 합성곱 신경망 (Convolutional Neural Network)

방 수 식 교수
(bang@tukorea.ac.kr)

한국공학대학교 전자공학부

2024년도 2학기
고급머신러닝실습 & 인공지능설계실습2

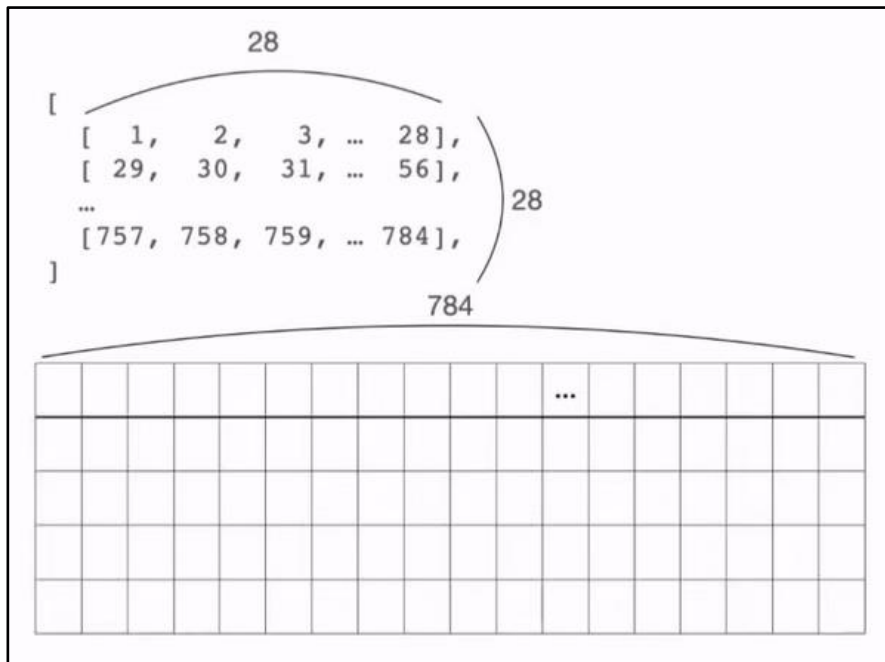
Artificial Neural Network의 문제점



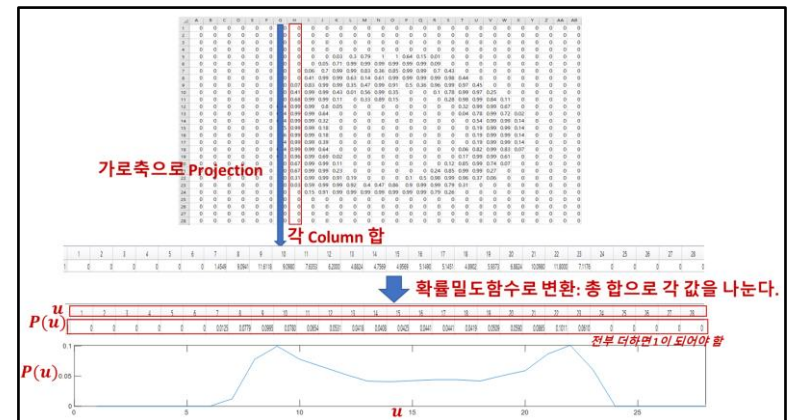
지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

■ 인공신경망의 문제

- 2차원 이상의 데이터를 1차원으로 Flatten
- Flatten 과정에서 공간정보 유실로 학습이 비효율적
- 공간정보를 유지하고 효율적인 학습을 위해 사람이 직접 특징을 추출하여, 입력에 넣기도 함



Flatten 과정



사람이 직접 특징 추출



특징 추출을 기계가 자동으로 할 수 있다면?

Convolutional Neural Network (CNN)

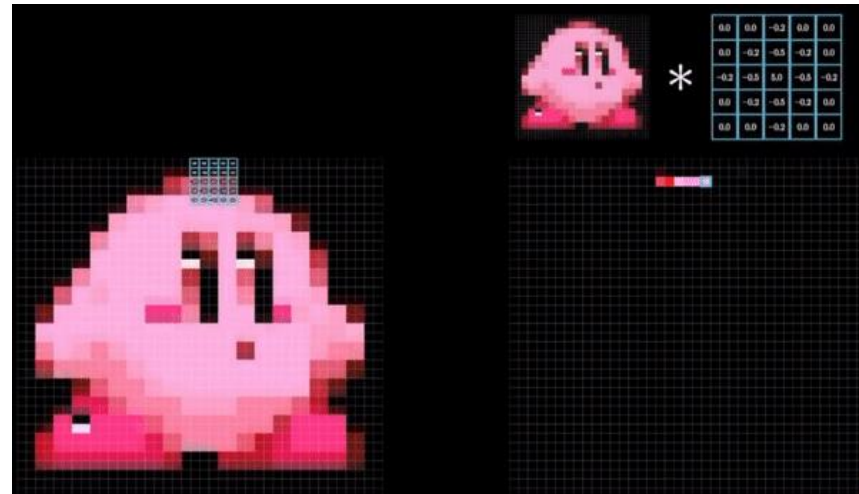


지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

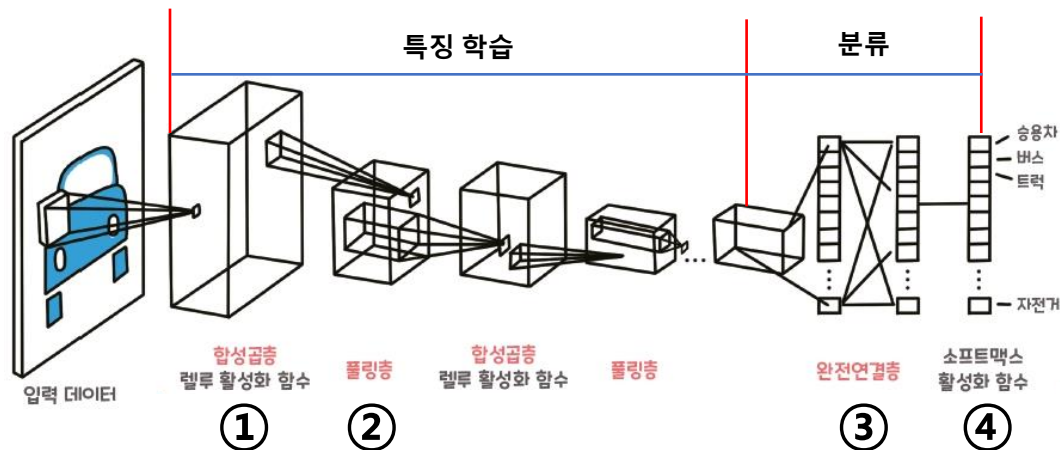
■ CNN (합성곱 신경망)

- 딥러닝의 가장 대표적인 방법, 주로 이미지 인식에 많이 사용됨
- 합성곱 신경망의 구조

- ① 합성곱층
- ② 풀링층
- ③ 완전연결층
- ④ 활성화함수



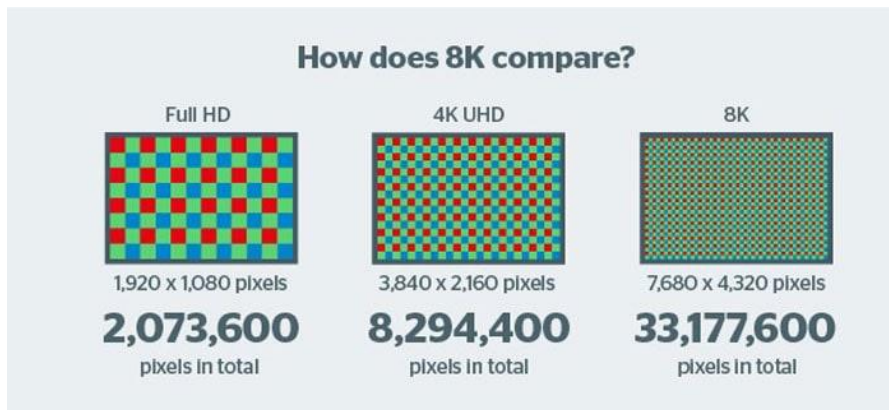
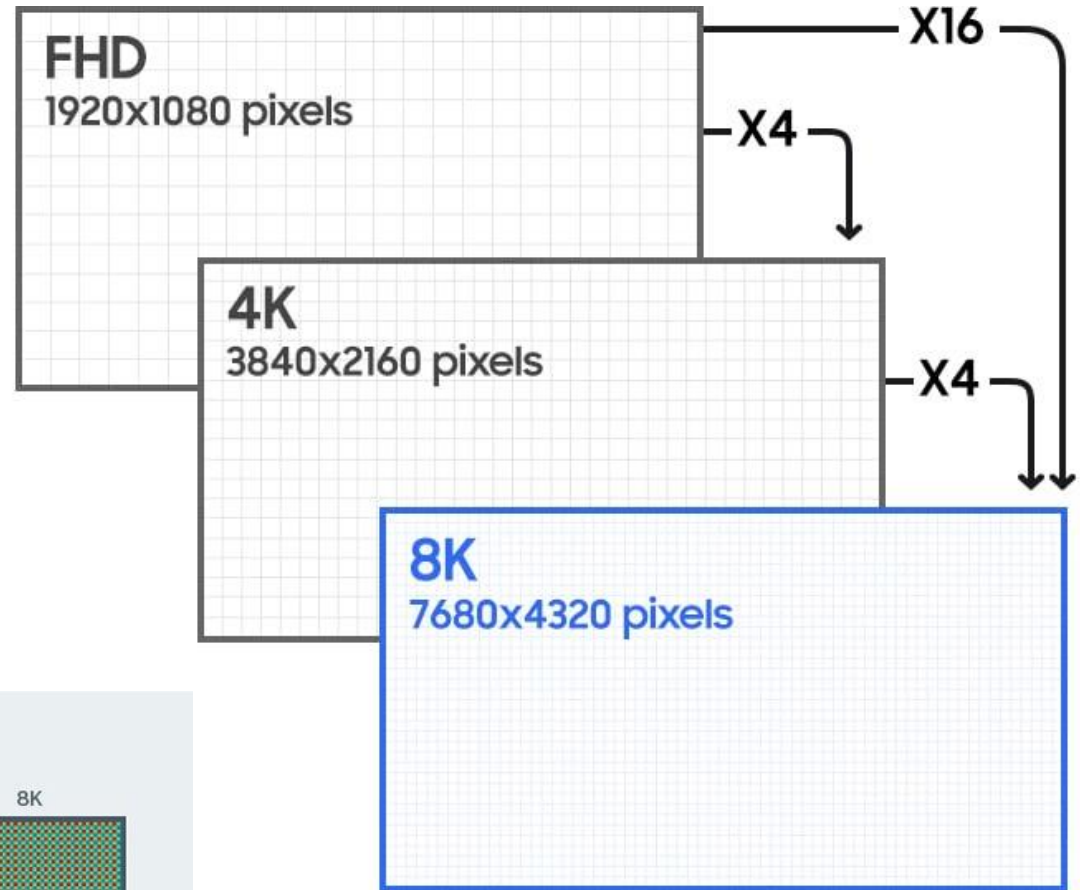
Convolution을 이용한 특징 추출



Image의 3요소



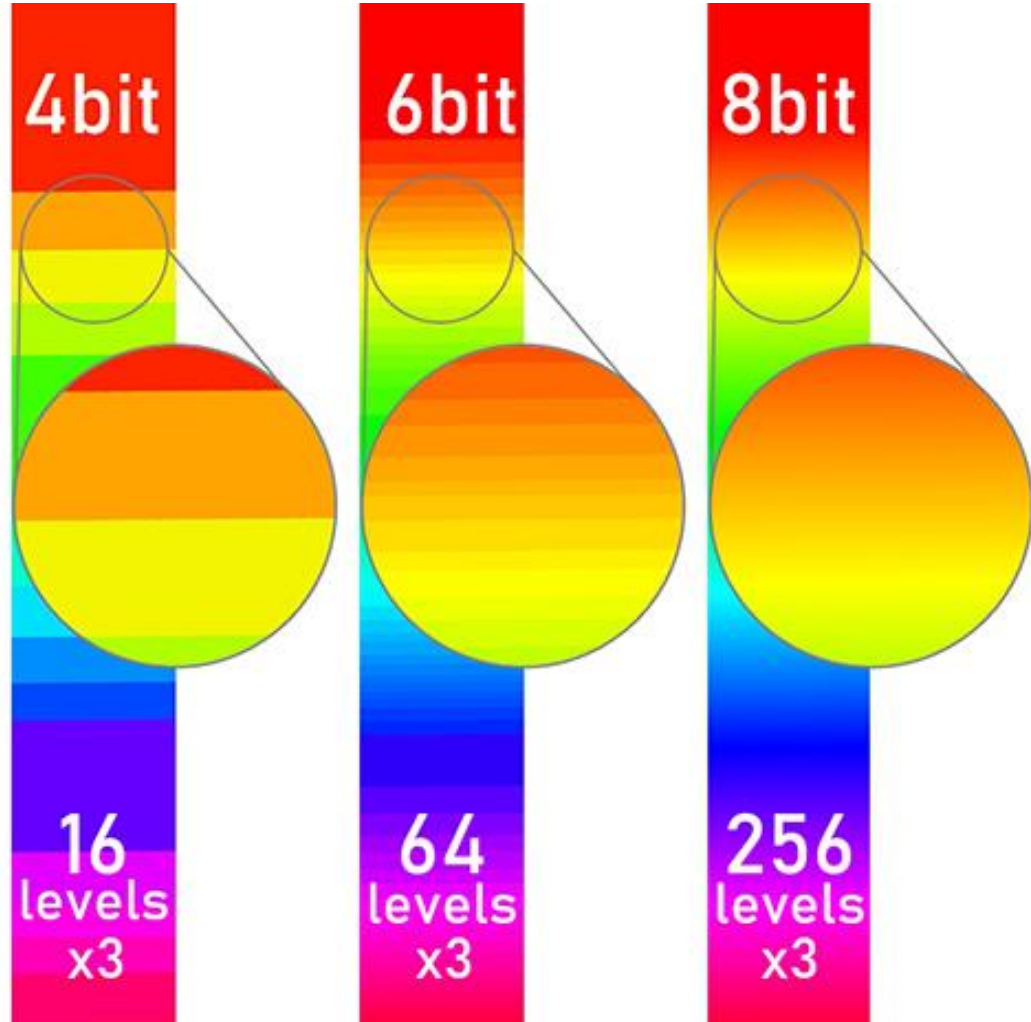
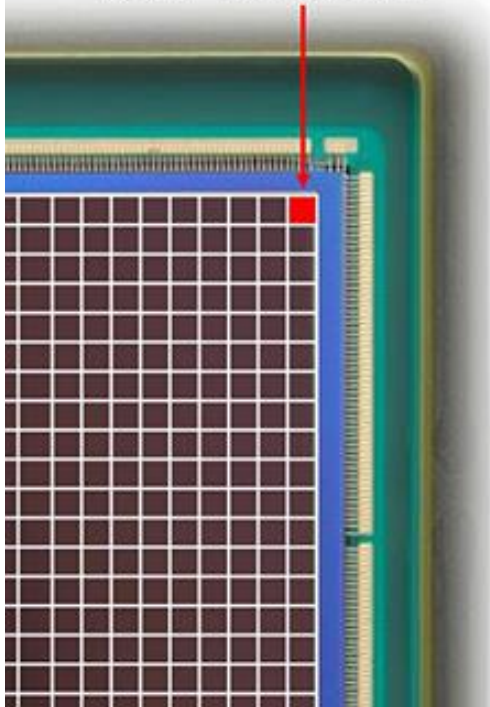
1. 이미지의 크기(Size)



2. 정밀도(Bit Depth)

Bit depth

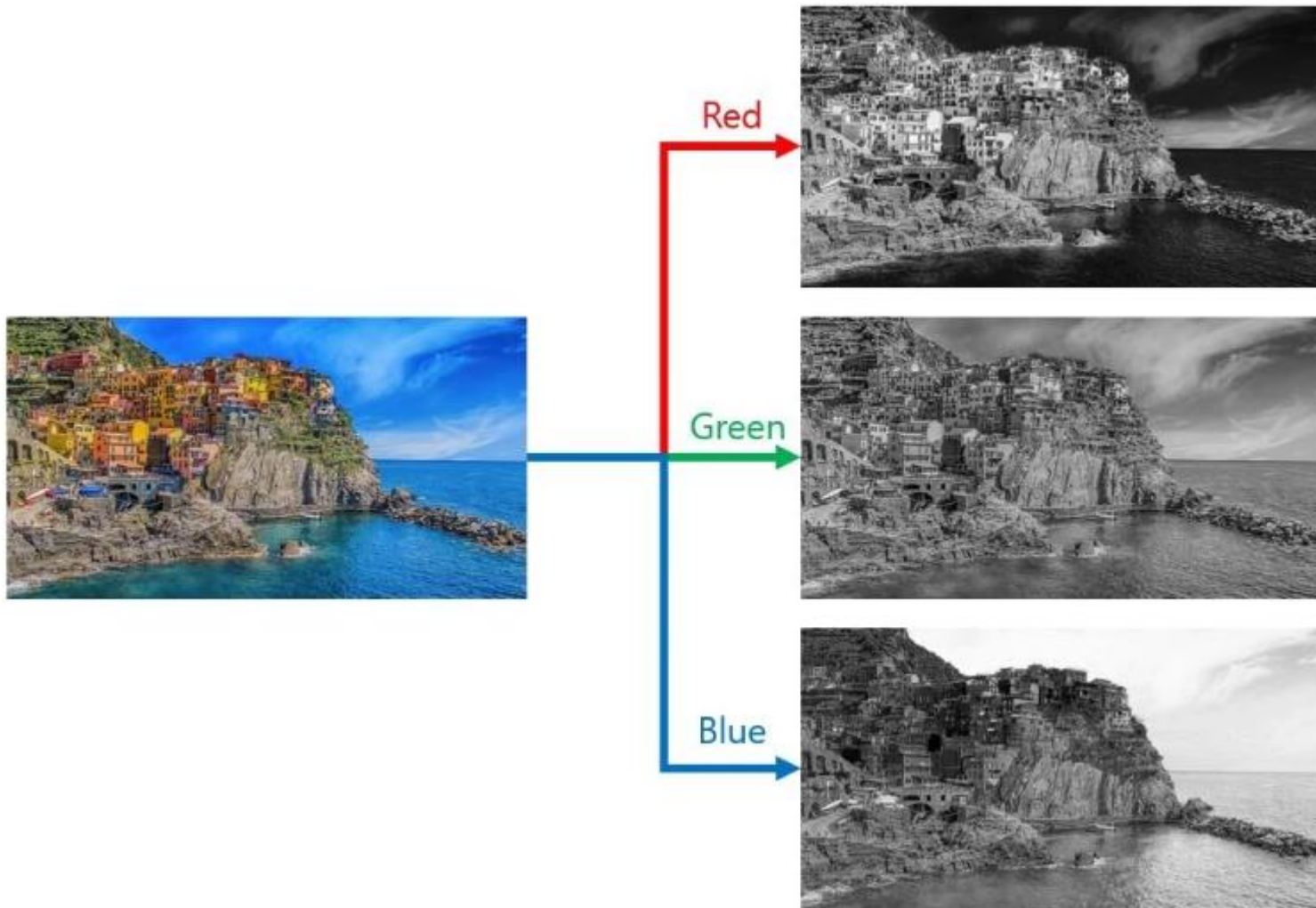
The number of colours and tones that 1 pixel can record



Image의 3요소



- 3. 채널(Channel)



Convolutional Layer

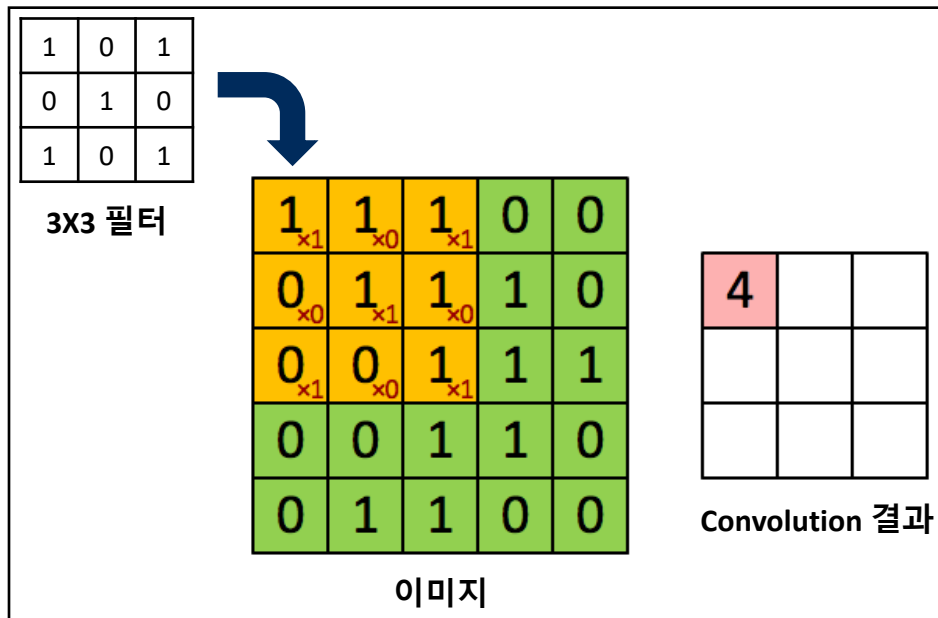


■ Convolutional Layer(합성곱 층)

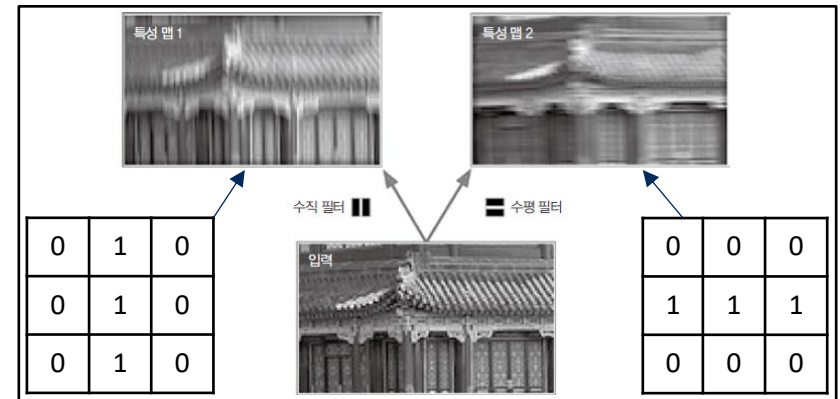
- CNN의 가장 중요한 구성 요소로, 입력의 특징을 추출

■ Single-Channel Image에서의 Convolutional Filter(필터)

- Convolution 연산에 사용되는 행렬로, 하나의 필터는 하나의 특성 맵 생성
- Filter 또한 하나의 weight로 동작하여, CNN의 학습 대상임 → **필터도 학습됨**



Convolution 수행 과정



각 필터에 따른 특성 맵

채널이 다수인 이미지라면?

Convolutional Layer



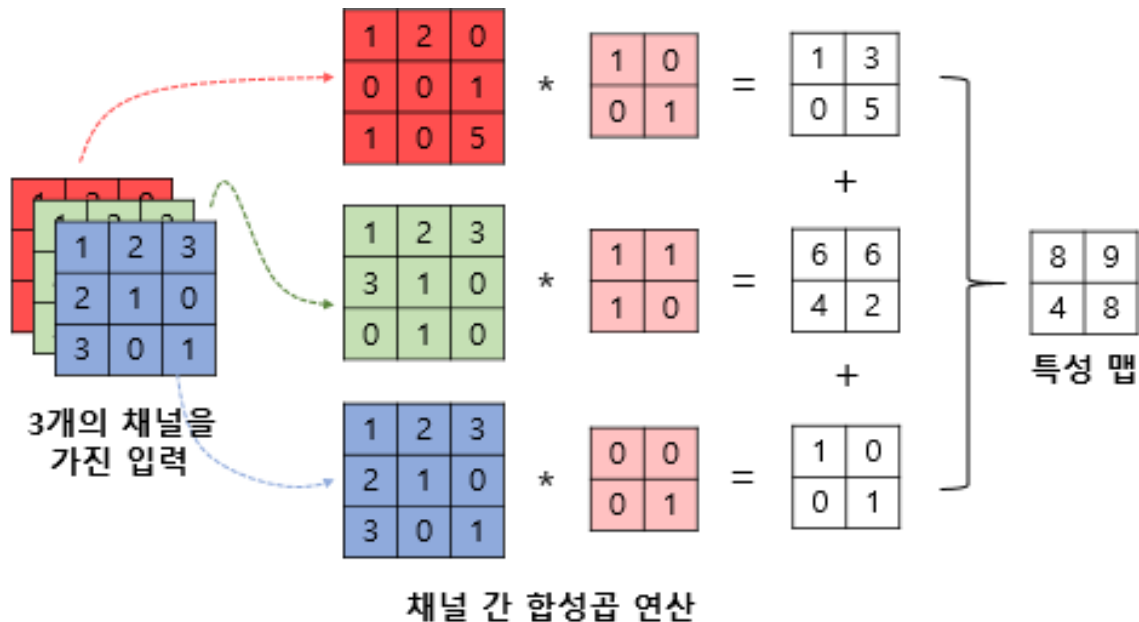
■ Multi-Channel에서의 Convolution

- 채널마다 필터를 배정

➢ N채널 이미지일 때, 필터도 N개 배정 => 필터를 3차원 배열로 생각해야 함.

- 채널 별 특성 맵을 모두 합해서 하나의 특성 맵으로

➢ 이미지 1개 당 특성 맵은 1개



Convolutional Layer



■ Multi-Channel에서의 Filter와 Feature Map

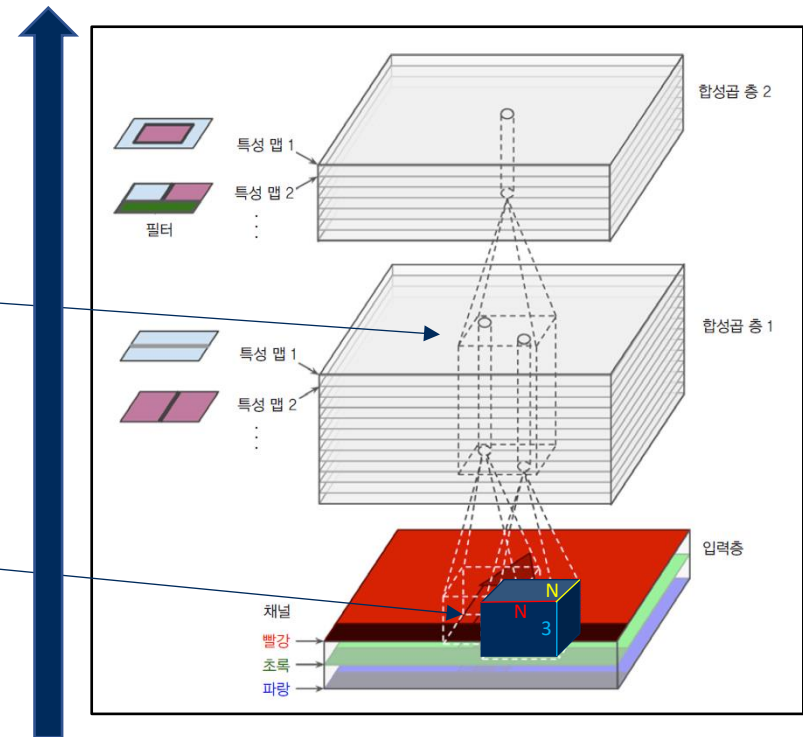
- Filter: 3차원 배열. Filter set: 4차원 배열
- Filter의 개수 만큼 Feature Map이 쌓임

이전 계층에서 K개의 Filter 사용
→ 현재 K개의 Feature Map 존재
→ Filter Size를 $M \times M$ 으로 설정
→ $M \times M \times K$ 의 Filter 1개로 Convolution

3차원 배열 필터를 1개의 필터로 간주

이미지: 3채널, Filter Size: $N \times N$ 설정
=> $N \times N \times 3$ 의 필터 1개로 Convolution

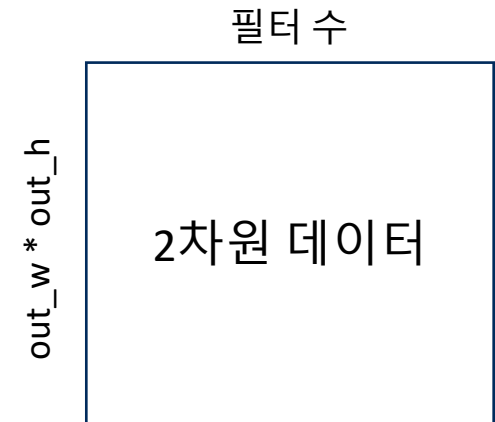
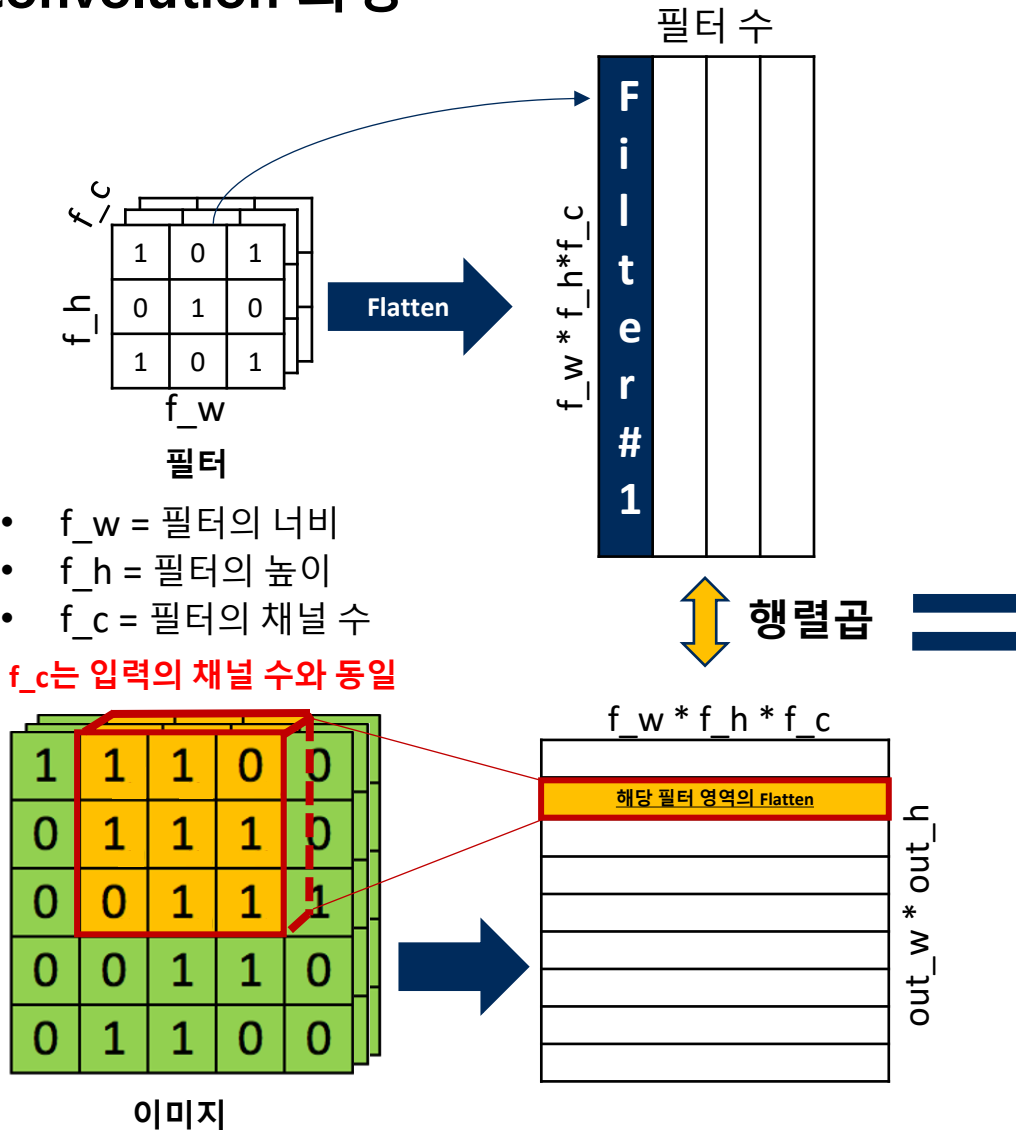
3차원 배열 필터를 1개의 필터로 간주



Coding for Convolutional Layer

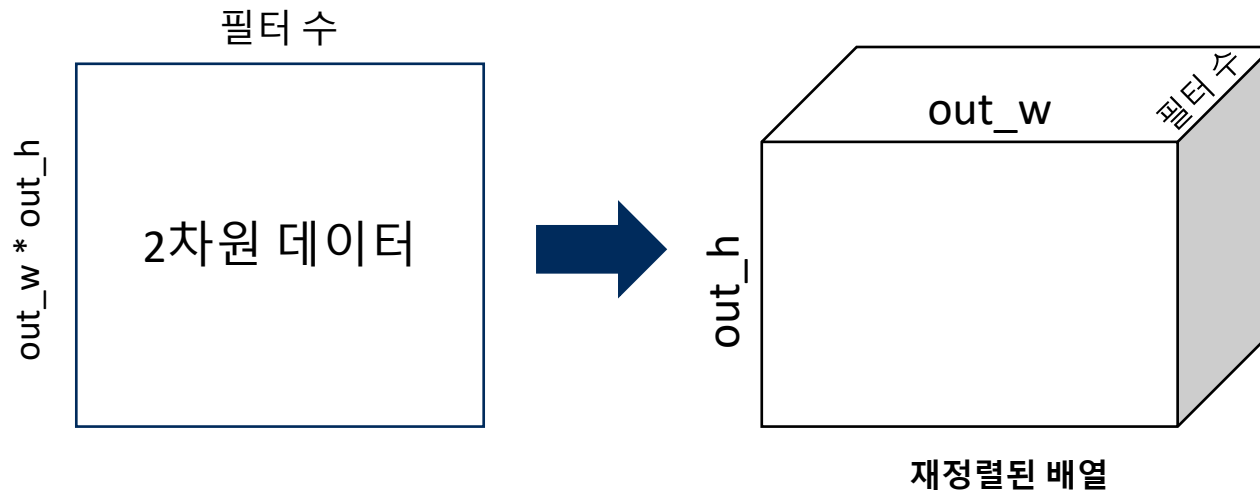


Convolution 과정



- out_w = 출력 이미지의 너비
- out_h = 출력 이미지의 높이

Convolution Layer의 출력



Convolution Layer의 역전파는?

- Convolution Layer의 순전파의 역순으로 진행
- 순전파 과정 : 합성곱 위치에 맞게 2차원 변경 → 행렬곱 → 출력 차원에 맞게 변경
- 역전파 과정 : legacy 이미지를 2차원 변경 → 미분 → 기존 입력 크기에 맞게 변경
- 미분 과정은 앞선 완전연결층과 동일 (행렬곱이기 때문에)

Convolutional Layer의 옵션



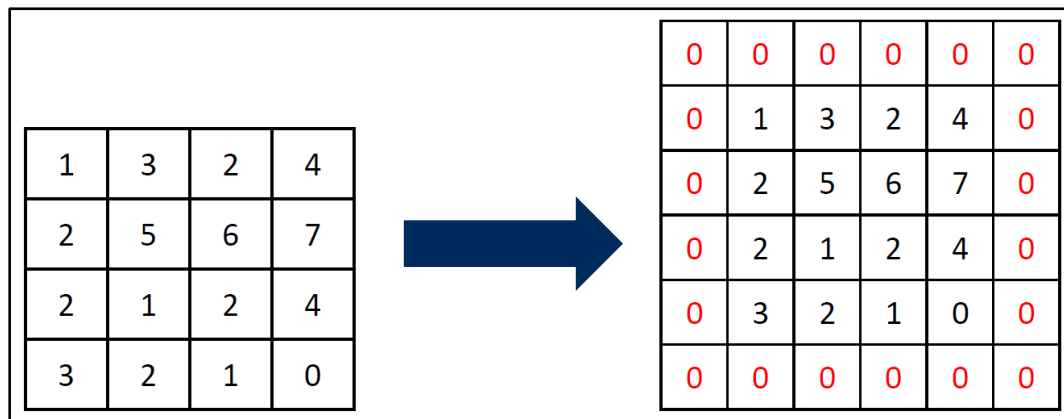
■ Zero Padding

- image 주위를 0으로 둘러주는 과정

Zero Padding이 왜 필요할까?

■ 이미지 데이터의 축소를 막기 위해

- 이미지가 Convolution 연산으로 인해 너무 작아지면 학습시킬 데이터가 부족
Edge pixel data를 충분히 활용하기 위해
- 중요한 정보가 모서리나 외곽에 있을 경우 모델의 성능이 떨어지는 것을 최소화



Zero Padding

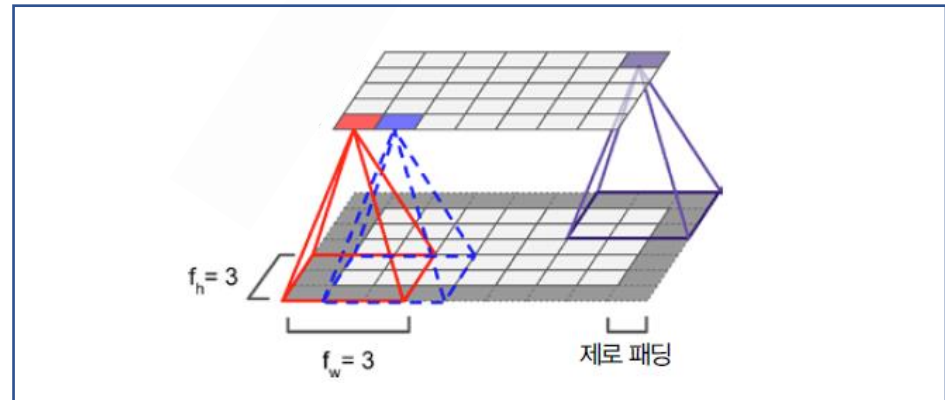
Convolutional Layer의 옵션



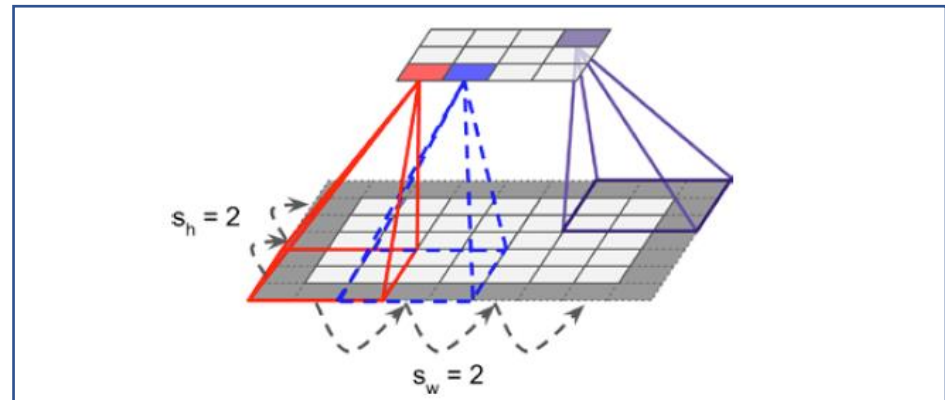
■ Stride

- 필터링 하는 간격 $\rightarrow$ 몇칸씩 움직일 것인가
- Stride는 출력 데이터의 크기를 조절하기 위해 사용됨

Stride 가 1인 경우



Stride 가 2인 경우



Convolutional Layer



■ Keras를 활용한 합성곱 층 구현

```
keras.layers.Conv2D(128, 3, activation="relu", padding="same")
```

①

②

③

④

① 128개의 Filters를 이용하여 Convolution 진행

② 3 by 3 사이즈의 Filter 사용

③ ReLU 활성화 함수 사용

④ Zero Padding 활성화

Stride는 기본값 1 사용

Pooling Layer



■ Pooling Layer

```
keras.layers.MaxPooling2D(2), #풀링 층 필터 크기 2 by 2
```

- 계산량, 메모리 사용량 등을 줄여 학습의 효율 ↑
- Overfitting 방지

■ Pooling 종류

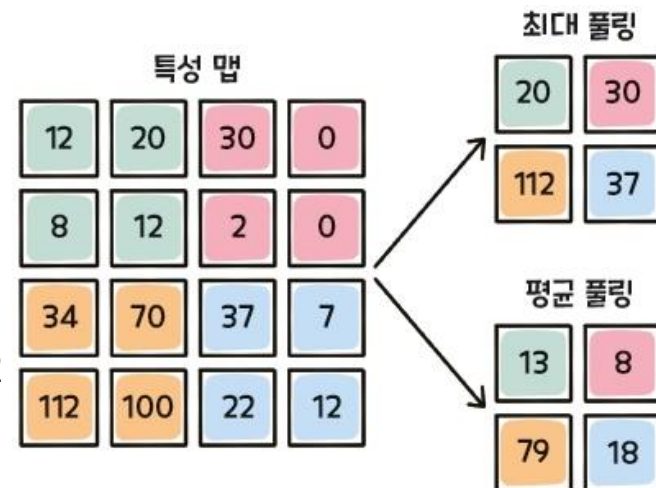
1) Max Pooling Layer

- 일반적으로 성능이 좋음
- 의미 없는 정보는 모두 제거, 큰 특징만 유지

2) Average Pooling Layer

- Max Pooling보다 정보 손실이 적음

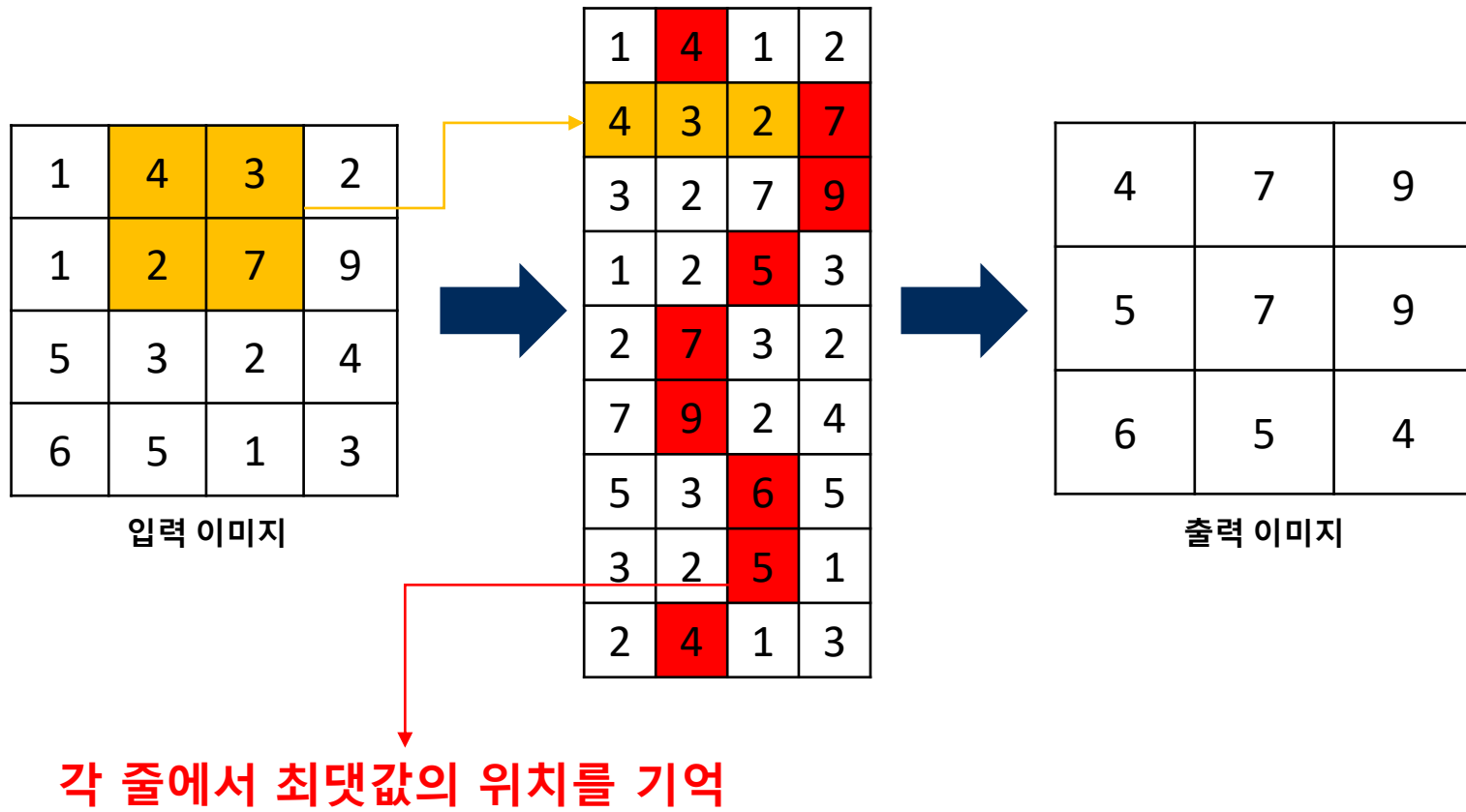
- 풀링 필터 크기: 2 by 2
- Stride: [2, 2]



Pooling Layer



- Max Pooling (예시 : Pooling Filter size 2 by 2 / Stride 1)



Pooling Layer



- Max Pooling

3	5	7
4	6	7
5	4	3

legacy 이미지



	3		
			5
			7
		4	
	6		
	7		
		5	
		4	
	3		



	3		
		6	7
4			3
5	4		

backward 이미지

Pooling 과정에서 기억해놓은 위치에 넣기 (나머지는 0)

■ 데이터셋

- keras의 MNIST 데이터셋 불러오기

```
from tensorflow import keras

mnist = keras.datasets.mnist
(X_train_full, Y_train_full), (X_test, Y_test) = mnist.load_data()
```

- Validation set, Train set으로 나누기 (총 5000개의 데이터만 사용)

```
X_valid, X_train = X_train_full[:500], X_train_full[500:5000]
Y_valid, Y_train = Y_train_full[:500], Y_train_full[500:5000]
```

MNIST 데이터 분류하기



■ 모델 학습

- 학습 방식 환경 설정

```
optimizer = keras.optimizers.SGD(lr = 0.0005)
model.compile(loss = "sparse_categorical_crossentropy",
              optimizer = optimizer,
              metrics=["accuracy"])
```

- 학습 진행

```
history = model.fit(X_train, Y_train, epochs = 100, batch_size = 128,
                  validation_data = (X_valid, Y_valid))
```

- 테스트셋을 이용하여 정확도 확인

```
model.evaluate(X_test, Y_test)
```

```
313/313 [=====] - 0s 1ms/step - loss: 0.1023 - accuracy: 0.9747
```

```
[0.10225518047809601, 0.9746999740600586]
```

MNIST 데이터 분류하기



■ 모델 생성

- `models.Sequential`을 이용하여 CNN 모델 생성

```
model = keras.models.Sequential([
    keras.layers.Conv2D(100,7,activation = "relu", strides = (2,2),
                        padding = "same", input_shape = [28,28,1]),
    keras.layers.MaxPool2D((2,2)),
    keras.layers.Flatten(),
    keras.layers.Dropout(0.25),
    keras.layers.Dense(units=64, activation="relu"),
    keras.layers.Dropout(0.25),
    keras.layers.Dense(units=10, activation="softmax"),
    ])
```

Drop-out 기법



■ Drop-out

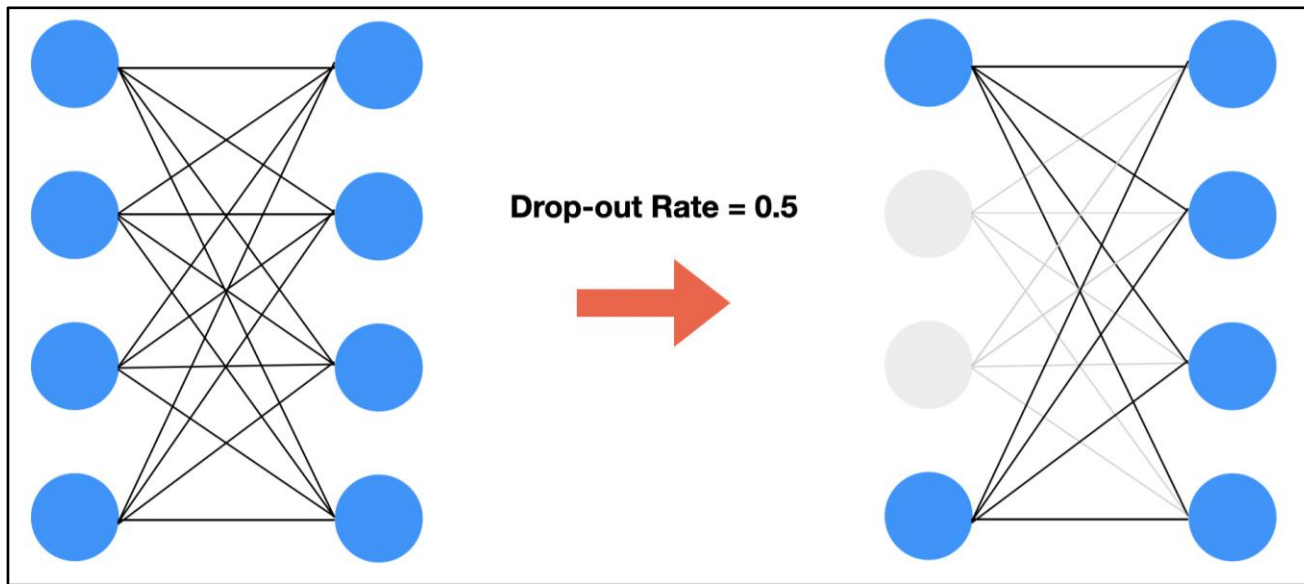
```
keras.layers.Dropout(0.25),
```

→ 25%의 확률로 노드 제거

- 서로 연결된 층에서 0~1 사이의 확률로 노드를 제거하는 기법
- 어떤 특정한 설명변수 Feature만을 과도하게 학습하는 것을 방지



Overfitting 방지



Drop-out

MNIST 데이터 분류하기

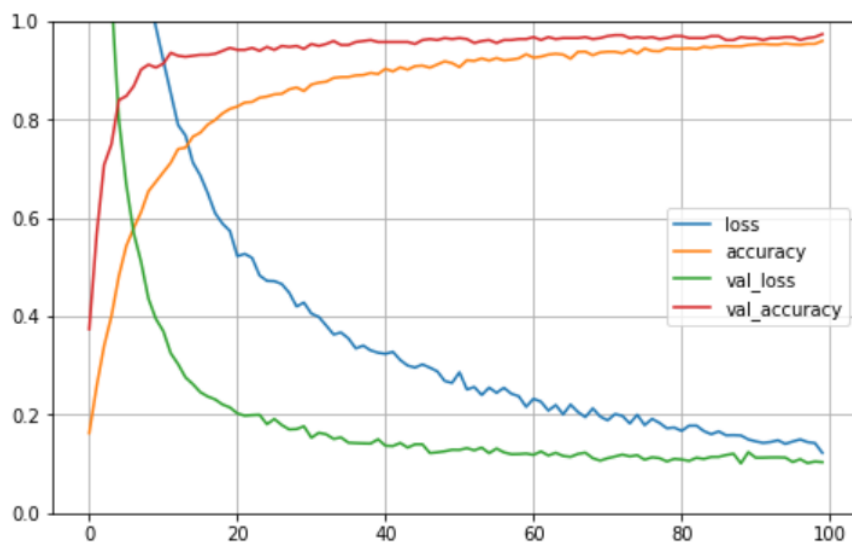


■ 예측 결과 확인

- 학습결과 그래프로 확인 (Epoch에 따른 Accuracy 및 Loss)

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

pd.DataFrame(history.history).plot(figsize=(8,5))
plt.grid(True)
plt.gca().set_ylim(0,1)
plt.show()
```



MNIST 데이터 분류하기

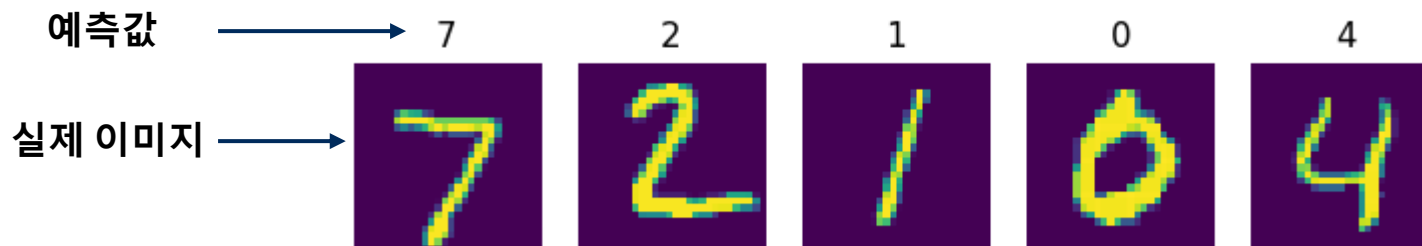


■ 예측 결과 확인

- 5개의 테스트셋 예측 결과 확인

```
X_new = X_test[:5]
Y_pred = np.argmax(model.predict(X_new), axis = -1)
print(Y_pred)

plt.figure(figsize=(7,2))
for index, image in enumerate(X_new):
    plt.subplot(1,5,index+1)
    plt.imshow(image)
    plt.axis('off')
    plt.title(Y_test[index], fontsize=12)
plt.show()
```



MNIST 데이터 분류하기



■ 예측 결과 확인

- 틀린 예측 확인

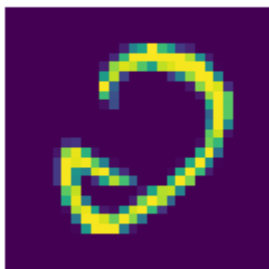
```
wrong_index_list = []
predict = model.predict(X_test)
for index in range(5000):
    if Y_test[index] != np.argmax(predict[index], axis = -1):
        wrong_index_list.append(index)

import random
new_index = random.sample(wrong_index_list, 5)
wrong_X_new = X_test[new_index]

plt.figure(figsize=(7,2))
for index, image in enumerate(wrong_X_new):
    plt.subplot(1,5,index+1)
    plt.imshow(image)
    plt.axis('off')

    temp_Actial = Y_test[new_index[index]]
    temp_Predict = np.argmax(predict[new_index[index]])
    title = "Actial : {} Predict : {}".format(temp_Actial, temp_Predict)
    plt.title(title, fontsize=8)
plt.show()
```

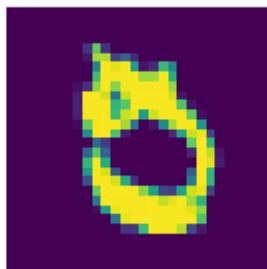
Actial : 2 Predict : 0



Actial : 9 Predict : 8



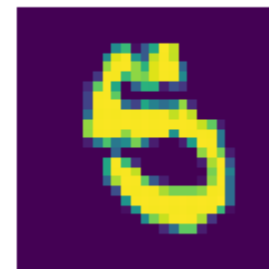
Actial : 5 Predict : 6



Actial : 8 Predict : 4



Actial : 5 Predict : 6



Chap.4 실습



각 실습문제에 대해 *code(+주석)*, *그래프*, *분석 내용* 등은 필수적으로 포함시킬 것

실습 #1) 케라스를 이용하여 CNN 구현

- fashion_mnist 데이터셋 활용

```
fashion_mnist = keras.datasets.fashion_mnist  
(X_train_full, Y_train_full), (X_test, Y_test) = fashion_mnist.load_data()
```

- 여러 Hyper-Parameter를 조절하여 정확도 향상
- Train set, Validation set으로 분할 (조건 : Train + Validation 개수 5000개로 제한)
- 학습 후 Test set을 이용하여 정확도 확인
- Epoch에 따른 Accuracy 및 정확도 확인
- 예측에 실패한 값 확인

Index별 클래스 이름

```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",  
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

각 실습문제에 대해 code(+ 주석), 그래프, 분석 내용 등은 필수적으로 포함시킬 것

실습 #2) CNN 모델 개선

- 모델 구조를 수정하여 자신만의 CNN 모델 만들기
- 실습 #1의 결과보다 높은 Test 정확도 구현
- Train set, Validation set으로 분할 (조건 : Train + Validation 개수 5000개로 제한)
- 학습 후 Test set을 이용하여 정확도 확인
- Epoch에 따른 Accuracy 및 정확도 확인
- 예측에 실패한 값 확인

실습 #3) 새로운 Data set에 적용

- 실습 #2의 모델에 새로운 Data set을 학습 (개수 제한 X) → input_shape 주의
- Test set 정확도 확인
- 실습 #2의 모델 개선 및 하이퍼 파라미터 변경을 통해 정확도 개선

```
from tensorflow import keras

cifar10 = keras.datasets.cifar10
(X_train_full, Y_train_full), (X_test, Y_test) = cifar10.load_data()

class_names = ["Airplane", "Car", "Bird", "Cat", "Deer",
               "Dog", "Frog", "Horse", "Ship", "Truck"]
```